CEI Asked Coding Questions

Problem statements with Python solutions

1. Valid Palindrome (LeetCode 125 – Easy)

Problem

A phrase is a palindrome if, after converting all uppercase letters to lowercase and removing all non-alphanumeric characters, it reads the same forward and backward. Given a string s, return true if it is a palindrome, otherwise false.

Examples

- Input: s = "A man, a plan, a canal: Panama" → Output: true
- Input: s = "race a car" → Output: false
- Input: s = " " → Output: true (empty string after cleaning)

Constraints

- 1 <= s.length <= 2 * 10^5
- s consists only of printable ASCII characters

Solution (Python 3)

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
        res = ""
        for i in s:
            if i.isalpha() or i.isdigit():
                i = i.lower()
                res+=i
        return res==res[::-1]
```

Complexity Time: O(n) | Space: O(n)

2. Group Anagrams (LeetCode 49 – Medium)

Problem

Given an array of strings strs, group the anagrams together. You can return the answer in any order.

Examples

- Input: strs = ["eat","tea","tan","ate","nat","bat"] → Output: [["bat"],["nat","tan"],["ate","eat","tea"]]
- Input: strs = [""] → Output: [[""]]
- Input: strs = ["a"] → Output: [["a"]]

Solution (Python 3)

```
class Solution:
    def groupAnagrams(self, strs: list[str]) -> list[list[str]]:
        groups = {}
        for i in strs:
            key = ''.join(sorted(i))

            if key not in groups:
                groups[key] = []
            groups[key].append(i)

        return list(groups.values())
```

Complexity Time: O(N · K log K) | Space: O(N · K) (N = number of strings, K = max string length)

3. URL Validation using Regex

Problem

Read a URL from the user and check whether it is valid using a regular expression (must start with http:// or https://, contain a valid domain with a TLD, and may have an optional path).

Solution (Python 3)

```
import re

url = input("Enter URL: ")

pattern = r'^https?://[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}(/\S*)?$'

if re.match(pattern, url):
    print("Valid URL")
else:
    print("Invalid URL")
```

Complexity Time: O(n) in the length of the URL

4. Longest Common Subsequence (LeetCode 1143 – Medium)

Problem

Given two strings text1 and text2, return the length of their longest common subsequence. If there is no common subsequence, return 0. A subsequence is formed by deleting some (or no) characters without changing the relative order of the remaining characters.

Examples

- Input: text1 = "abcde", text2 = "ace" → Output: 3 ("ace")
- Input: text1 = "abc", text2 = "abc" → Output: 3
- Input: text1 = "abc", text2 = "def" → Output: 0

Constraints

- 1 <= text1.length, text2.length <= 1000
- text1 and text2 consist of only lowercase English characters

Solution (Python 3)

```
class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m = len(text1)
        n = len(text2)

        dp = [[0]*(n+1) for _ in range(m+1)]

        for i in range(1,m+1):
            for j in range(1,n+1):
                if text1[i-1] == text2[j-1]:
                    dp[i][j]=1+dp[i-1][j-1]
                else:
                    dp[i][j]=max(dp[i][j-1],dp[i-1][j])

        return dp[m][n]
```

Complexity Time: O(m · n) | Space: O(m · n)

5. Coin Change II (LeetCode 518 – Medium)

Problem

Given an integer array coins of different denominations and an integer amount, return the number of combinations that make up that amount. If it cannot be made up, return 0. You have an infinite number of each kind of coin.

Examples

- Input: amount = 5, coins = [1,2,5] → Output: 4
- Input: amount = 3, coins = [2] → Output: 0
- Input: amount = 10, coins = [10] → Output: 1

Constraints

- 1 <= coins.length <= 300
- 1 <= coins[i] <= 5000
- All values of coins are unique
- 0 <= amount <= 5000

Solution (Python 3)

```
class Solution:
    def change(self, amount: int, coins: list[int]) -> int:
        dp = [0] * (amount+1)
        dp[0] = 1
        for i in coins:
            for j in range(i,amount+1):
                dp[j] += dp[j-i]
        return dp[amount]
```

Complexity Time: O(amount · len(coins)) | Space: O(amount)

6. Game of Life (LeetCode 289 – Medium)

Problem

Given an m x n board where each cell is live (1) or dead (0), update the board to its next state by applying Conway's Game of Life rules simultaneously: (1) a live cell with fewer than 2 live neighbors dies; (2) a live cell with 2 or 3 live neighbors lives on; (3) a live cell with more than 3 live neighbors dies; (4) a dead cell with exactly 3 live neighbors becomes live. Update in place; nothing is returned.

Examples

- Input: board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]] → Output: [[0,0,0],[1,0,1],[0,1,1],[0,1,0]]
- Input: board = [[1,1],[1,0]] → Output: [[1,1],[1,1]]

Constraints

- 1 <= m, n <= 25
- board[i][j] is 0 or 1

Solution (Python 3)

```
class Solution(object):
    def gameOfLife(self, board):
        m = len(board)
        n = len(board[0])

        for i in range(m):
            for j in range(n):
                live = 0

                # Count live neighbors
                for x in range(max(0, i - 1), min(m, i + 2)):
                    for y in range(max(0, j - 1), min(n, j + 2)):
                        live += board[x][y] & 1

                # Exclude the current cell
                live -= board[i][j] & 1

                # Store the next state in the second bit
                if live == 3 or (live == 2 and board[i][j] == 1):
                    board[i][j] |= 2

        # Update all cells to their next state
        for i in range(m):
            for j in range(n):
                board[i][j] >>= 1
```

Complexity Time: O(m · n) | Space: O(1) – in place, the next state is stored in the second bit of each cell

7. Minimum Swaps to Arrange a Binary Grid (LeetCode 1536 – Medium)

Problem

Given an n x n binary grid, in one step you can swap two adjacent rows. A grid is valid if all cells above the main diagonal are zeros. Return the minimum number of steps needed to make the grid valid, or -1 if it cannot be made valid.

Examples

- Input: grid = [[0,0,1],[1,1,0],[1,0,0]] → Output: 3
- Input: grid = [[0,1,1,0],[0,1,1,0],[0,1,1,0],[0,1,1,0]] → Output: -1
- Input: grid = [[1,0,0],[1,1,0],[1,1,1]] → Output: 0

Constraints

- n == grid.length == grid[i].length
- 1 <= n <= 200
- grid[i][j] is either 0 or 1

Solution (Python 3)

```
class Solution(object):
    def minSwaps(self, grid):
        n = len(grid)
        trailing_zeros = []

        # Count trailing zeros in each row
        for row in grid:
            count = 0
            for j in range(n - 1, -1, -1):
                if row[j] == 0:
                    count += 1
                else:
                    break
            trailing_zeros.append(count)

        swaps = 0

        for i in range(n):
            required = n - 1 - i
            j = i

            # Find a row with enough trailing zeros
            while j < n and trailing_zeros[j] < required:
                j += 1

            # No suitable row exists
            if j == n:
                return -1

            # Move the row upward using adjacent swaps
            while j > i:
                trailing_zeros[j], trailing_zeros[j - 1] = (
                    trailing_zeros[j - 1], trailing_zeros[j]
                )
                swaps += 1
                j -= 1

        return swaps
```

Complexity Time: $O(n^2)$ | Space: $O(n)$